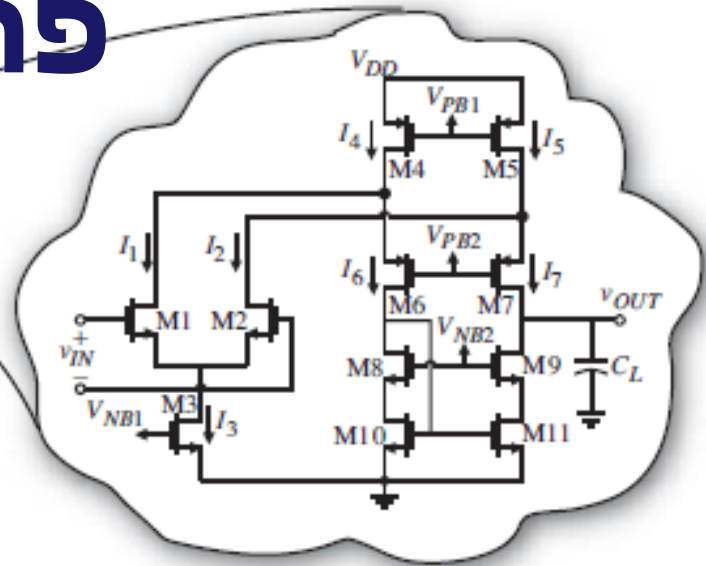


בדיקת מערכות ספרתיות ואימות תכנון

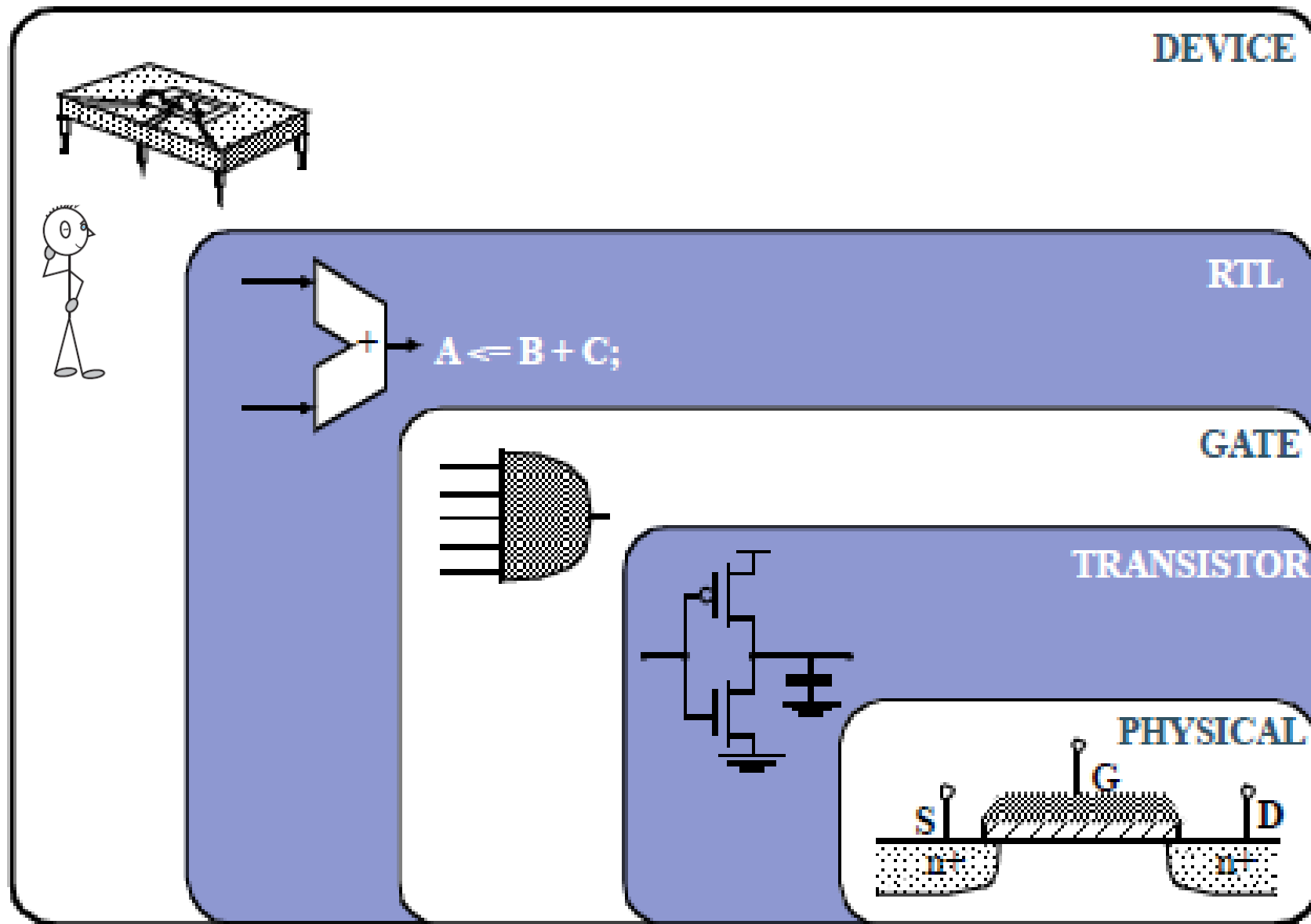
פרק 1

SPECIFICATIONS

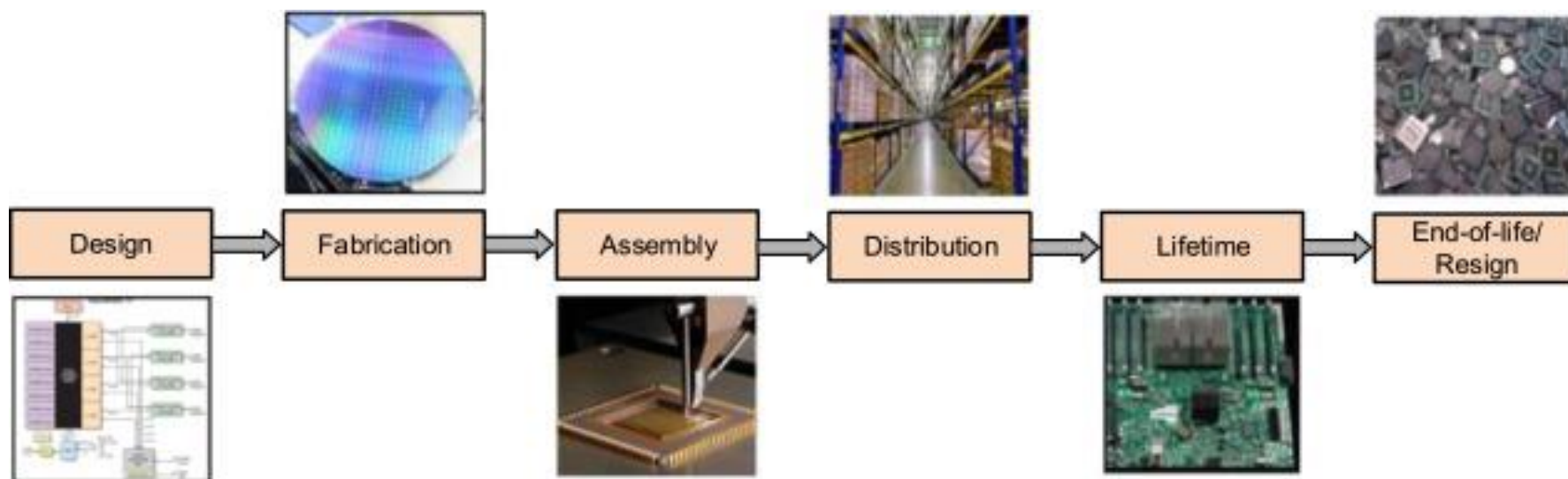


שילבי פיתוח VLSI

רמות אבסטרקציה



חיים של שבב



איך זה עובד?

1950's

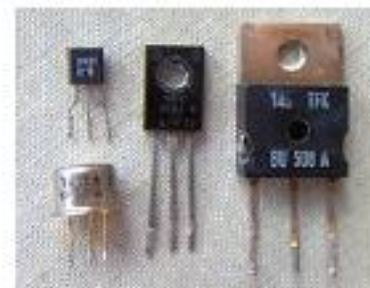
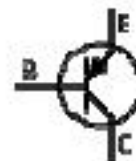
Digital Design
Let's Build An AND!

Draw Schematic

Select Components

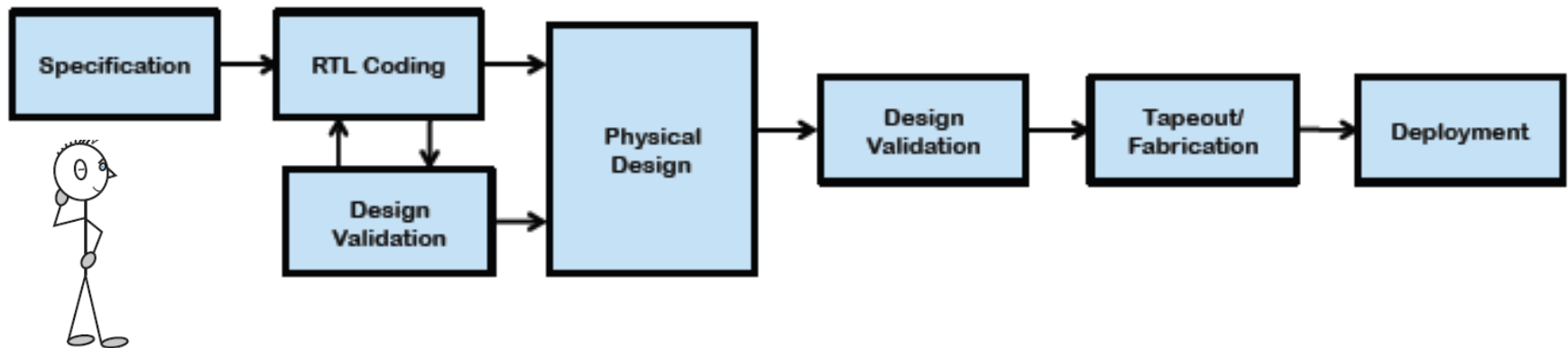
Implement

Idea

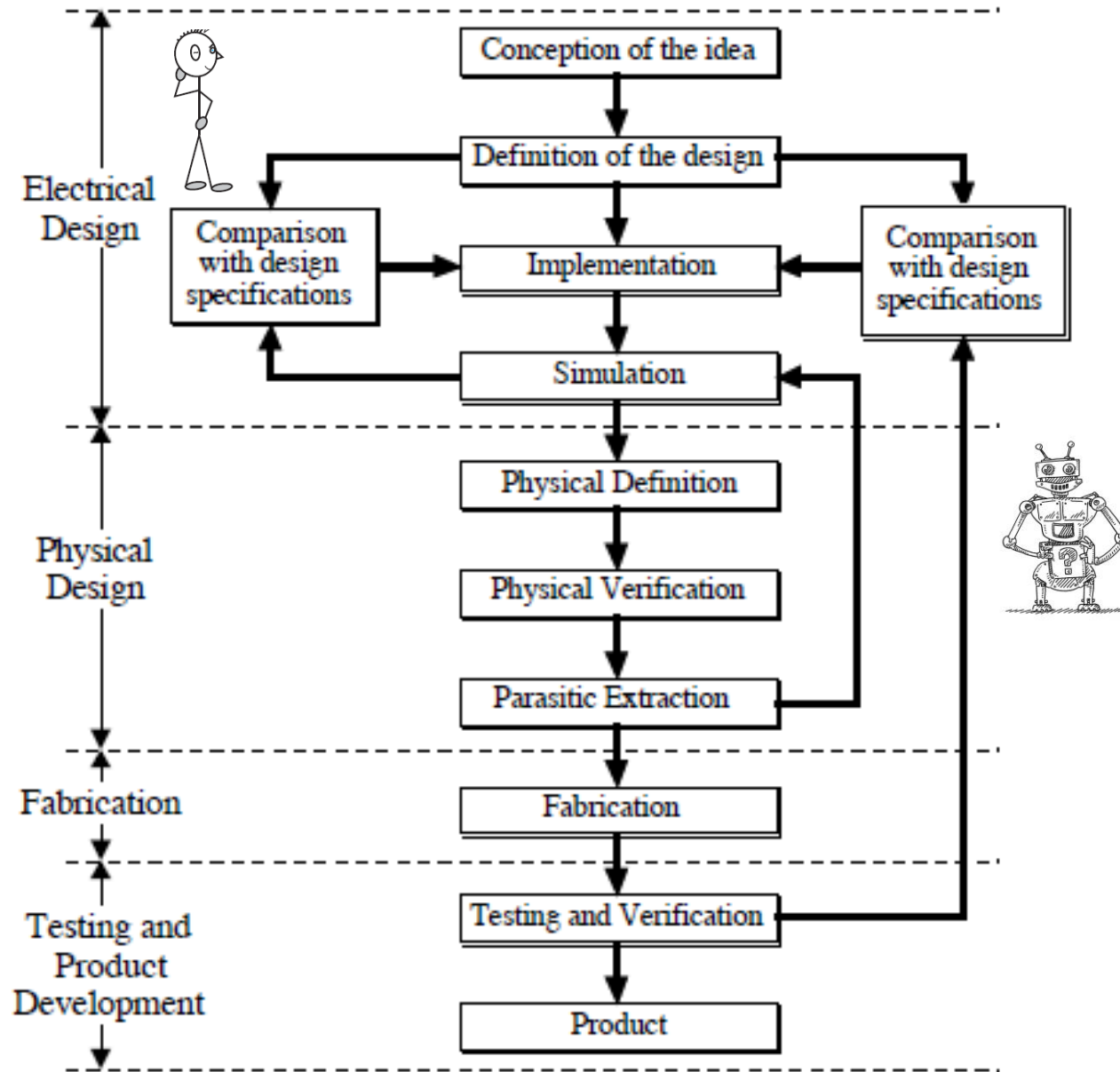


תהליך תכנון – Design Flow

תהליך תכנון VLSI מתחיל ממפרט רשמי טכני של השבב ועובר שורה של צעדים, ובסופו של דבר מיוצרים שבבים ארוזים



שלבי תכנון ובדיקות



Specification -- מפרט רשמי



Proposed by the project team



Product specification



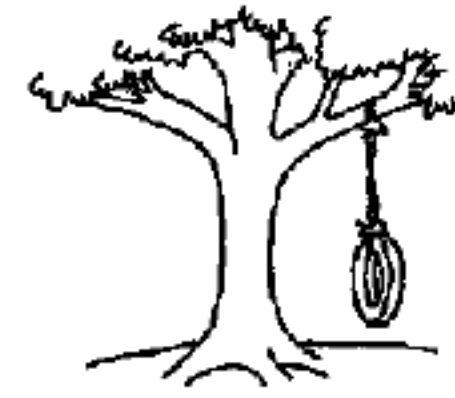
Product design by senior analyst



Product after implementation



Product after acceptance by user



What the user wanted

Specification – מפרט טכני רשמי

1. הצעד הראשון של תהליך התכנון הוא להגדיר את המפרט של המוצר

2. רמת ייצוג גבוהה של המערכת

3. גורמים שנלקחים בחשבון:

✓ ביצועים

✓ פונקציונליות

✓ מימד פיזי

✓ טכניקת תכנון

✓ טכנולוגית ייצור

4. זוהי פשרה בין דרישות השוק, כדאיות טכנולוגית וכלכלית



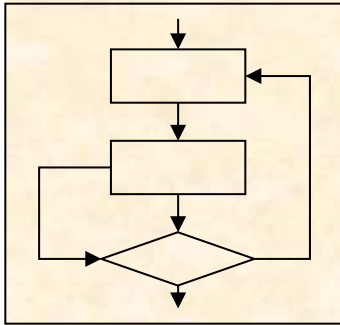
מפרט טכני רשמי (המשך)



בסוף הגדרת המפרט, נקבעים:

1. גודל
2. מהירות
3. הספק
4. פונקציונליות של רכיב ה-VLSI
5. הארכיטקטורה הבסיסית שמכילה:
 - ✓ יחידת נקודה צפה
 - ✓ מספר ALU
 - ✓ מספר והמבנה של חיבוריות
 - ✓ גודל ה-CACHE וכו' ✓

תכנון פונקציונאלי



1. יחידות תפקוד ראשיות של הרכיב מוגדרות

2. דרישות ההקשר בין היחידות מוגדרות

3. שטח, הספק ופרמטרים נוספים של כל יחידה מוגדרים

4. האיפיונים ההתנהגותיים של הרכיב מוגדרים

✓ הרעיון העיקרי הוא ציון התנהגות במושגים: קלט/פלט

✓ התזמון של כל יחידה

✓ המבנה הפנימי עוד לא מוגדר

תכנון פונקציונאלי (המשך)

5. התוצאה של תכנון פונקציונאלי הוא בדרך כלל דיאגרמת תזמון או יחסים אחרים בין יחידות

6. תכנון פונקציונאלי מספק אמוציה מהירה של המערכת, ומאפשר בדיקה מהירה של הרכיב

תכנון לוגי -- Logic Design

```
input in;
output out;
output [7:0] counter;

reg [7:0] counter;

wire counter_overflow = (counter == 8'hff);
wire counter_underflow = (counter == 8'h00);

always @(posedge clk or negedge xreset)
begin
    if (xreset == 0)
        counter <= 0;
    else if (en && in && !counter_overflow)
        counter <= counter + 1;
    else if (en && !in && !counter_underflow)
        counter <= counter - 1;
    else
        counter <= counter;
end
```

תכנון לוגי מכיל:

1. ביטויים בוליאניים,
2. העברת בקרה,
3. רוחב המילה,
4. הקצעת אוגרים, וכו'.

התוצאה נקראת RTL (Register Transfer Level) שמתבטא
בשפות כמו VHDL ו Verilog

תיאור זה ממומש בסימולציה ואימות.

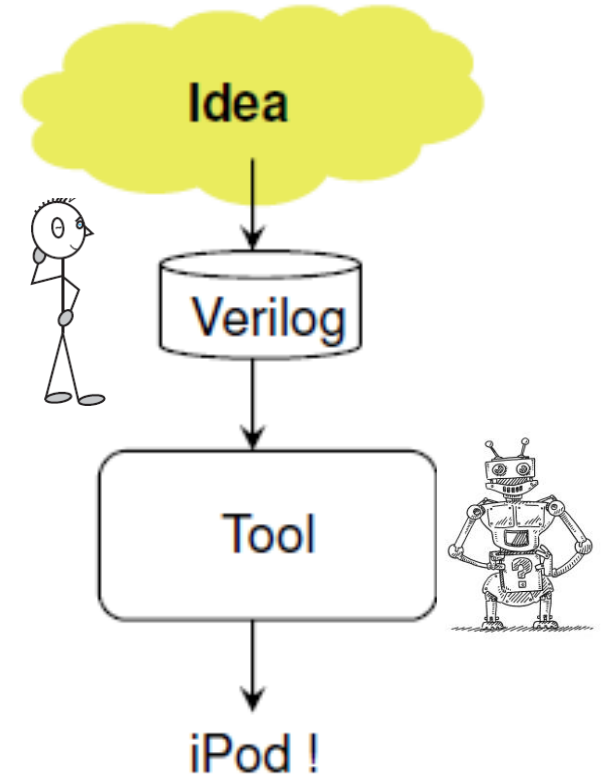
VHDL

Digital Design
Let's Build An XYZ!

"Write HDL" - Designer Activity

"Select Components" - Tool activity (*synthesis*)

"Implement" - Tool activity



תיאור חומרה

- "קוד" אתה כותב בשפה לתיאור החומרה לא נותן תוכנית כתוצאה!
-אתה מתאר חומרה -כלומר, משהו שבסופו של דבר יכול להיות מיושם כמעגלים אמיתיים
- יש נקודות משותפות בין HDL ותכנות מקבילי -כלומר, הכל קורה "בו זמנית"
-אבל נא לא לקחת את האנלוגיה רחוק מדי

מבוא ל-VERILOG

- Modules define circuit components.
- Instantiation defines hierarchy of the design.

keywords

```
module addr_cell (a, b, cin, s, cout);  
    input  a, b, cin;  
    output s, cout;  
  
    module body  
    endmodule  
  
module adder (A, B, S);  
    addr_cell ac1 ( ... connections ... );  
  
endmodule
```

name

port list

port declarations (input, output, or inout)

Instance of addr_cell

1-bit full adder : דוגמא

```
// Verilog one-bit adder slice - Structural description

module full-adder           // module name
    (a_one,                 // module ports
     a_two,
     c_in,
     s,
     c_out);

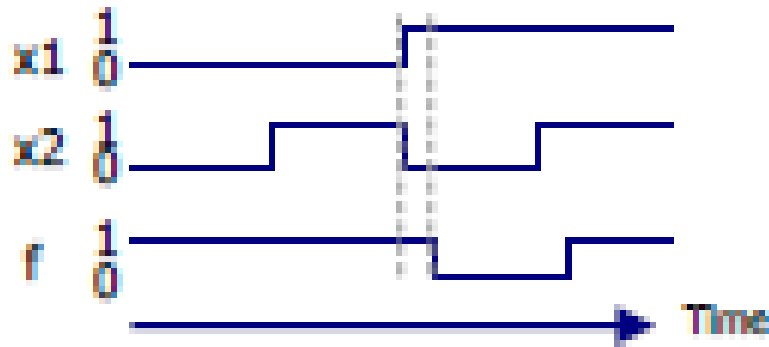
    input a_one, a_two;      // declare ports
    input c_in;
    output s;
    output c_out;

    wire s;                  // declare wires

    assign s = a_one ^ a_two ^ c_in;
    wire c_out = (a_one & a_two) | (a_one & c_in) | (a_two & c_in);

endmodule
```


איך אנחנו יודעים שהשבב שלנו באמת עובד?



פונקציונלית סימולציה

השיטה:

1. המהנדס מספק ערכי קלט

2. סימולטור פונקציונלי משתמש בערכים אלו
במשוואות

3. סימולטור פונקציונלי מייצר ערכי פלט, טבלאות אמת או
דיאגרמות תזמונים

4. המהנדס בוחן את התפוקה כדי לאמת את הרכיב

Structural vs. Behavioral Circuit Specification

- **מפרט מבני**

➤ מורכב רק מהפרימיטיבים או הגדרות של מודולים המופעים שלהם

➤ מערכת דיגיטלית מתוארת בצורה היררכית של הקישוריות בין המודולים / הפרימיטיבים אלה

- **מפרט התנהגותי**

➤ משתמש בביטויים אריתמטיים, השמות פרוצדורליים או פקודות אחרות של Verilog

Behavioral Specification

“Behavioral” example:

Decoder (output x0,x1,x2,x3;

inputs a,b)

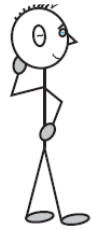
```
{  
    case [a b]  
        00: [x0 x1 x2 x3] = 0x1;  
        01: [x0 x1 x2 x3] = 0x2;  
        10: [x0 x1 x2 x3] = 0x4;  
        11: [x0 x1 x2 x3] = 0x8;  
    endcase;  
}
```

Structural Specification

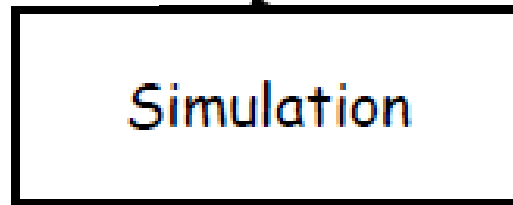
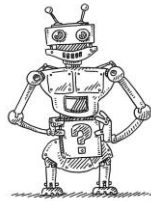
“Structural” example:

```
Decoder(output x0,x1,x2,x3;  
          inputs a,b)  
{  
    wire abar, bbar;  
    inv(bbar, b);  
    inv(abar, a);  
    and(x0, abar, bbar);  
    and(x1, abar, b);  
    and(x2, a, bbar);  
    and(x3, a, b);  
}
```

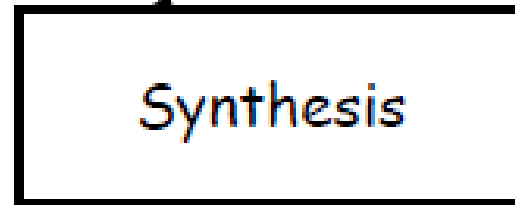
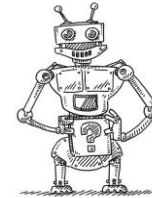
Specification HDL



Hierarchically defines
structure and/or function
of circuit.

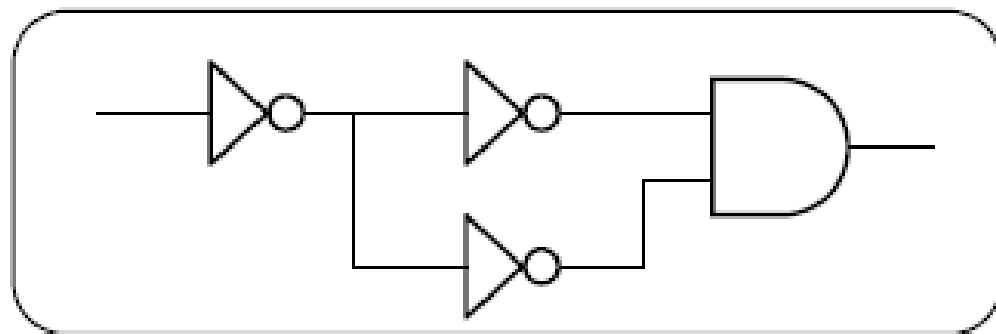


Verification: Does the design
behave as required with regards
to function, timing, and power
consumption?

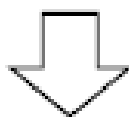


Maps specification to
resources of implementation
platform (FPGA or custom
silicon).

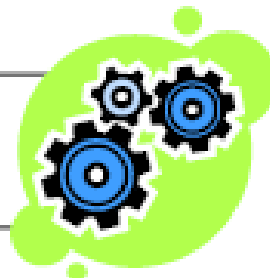
Simulation in Verilog



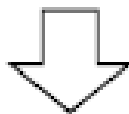
Verilog Model
of a hardware circuit
(many gates)



Modelsim
Verilog Simulator



Simulator creates the
illusion of parallel
activities in model



Sequential
Machine

Simulator is a software
program running on
a PC

Simulation in Verilog

סימולצית HDL שונה מריצת C

✓ חייבת לבטא מקביליות (דברים קורים בו זמנית)

✓ חייבת למדל זמן

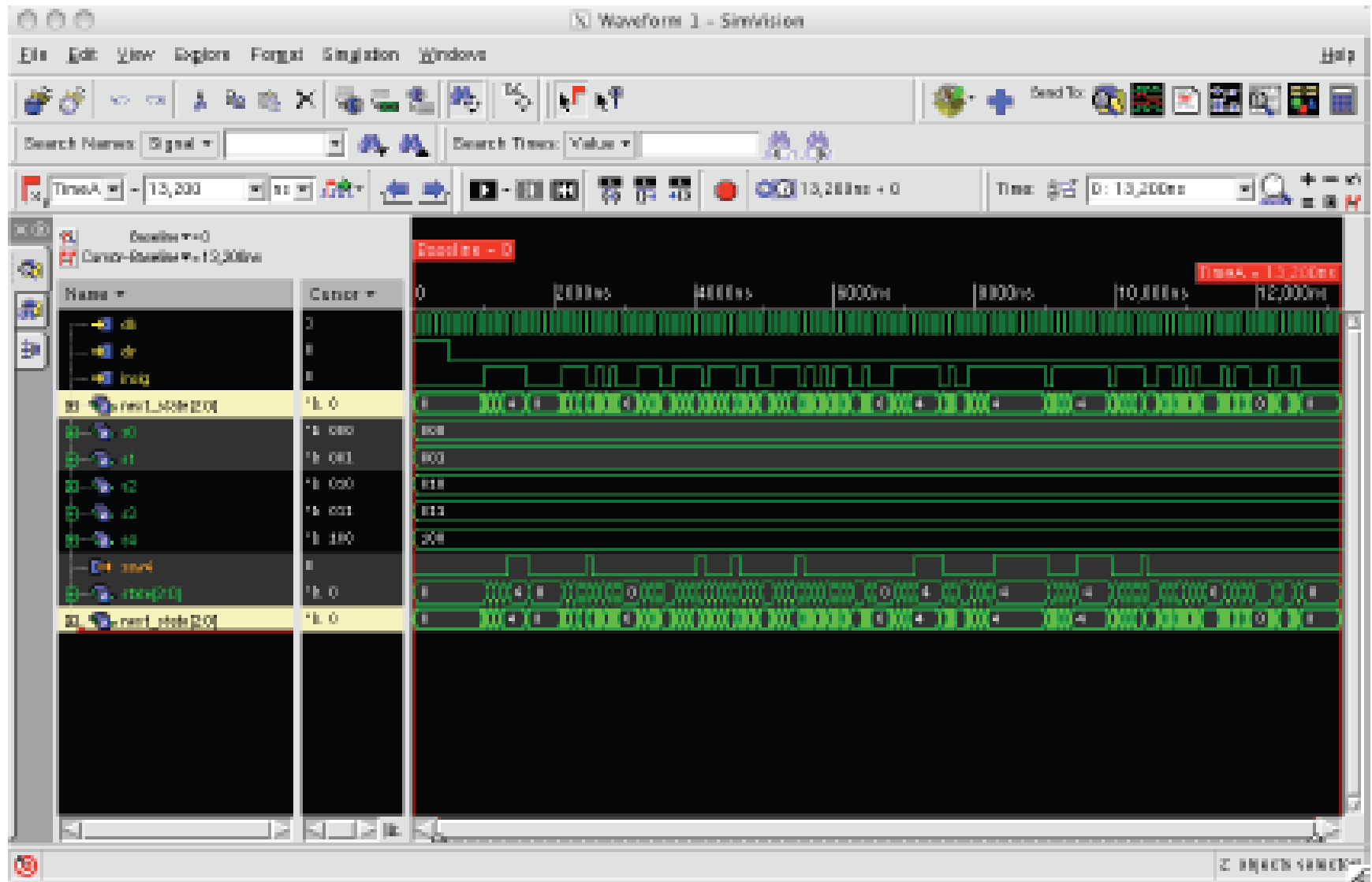
✓ חייבת לטפל במודלים לא סטנדרטיים (X, Z)

x: unknown logic value ➤

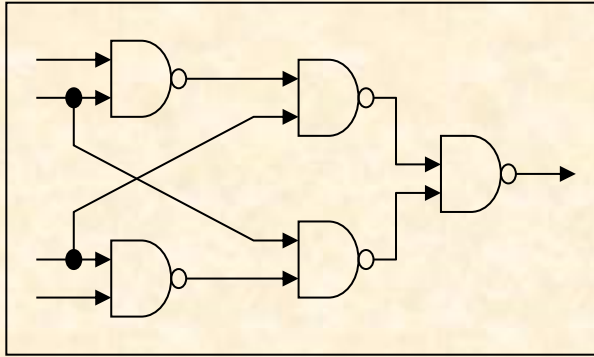
z: high-impedance state ➤

✓ חייבת לטפל במודולים ולא בפונקציות

Simulation in Verilog: Waveform



תכנון מעגלים -- Circuit Design



1. מטרת תכנון המעגלי היא לפתח ייצוג מעגל שמבוסס על תכנון לוגי.

2. ביטויים בוליאניים ניתן להמיר למעגלים ידי לקיחה בחשבון את גורם המהירות ואת דרישות הספק של התכנון המקורי.

תכנון מעגלים (המשך)

3. תכנון המעגלים כולל השערים ,

טרנזיסטורים, ה- interconnect
והתוצאה נקראת netlist.

4. סימולציה של מעגל מאמתת
את נכונותו והתזמונו של הרכיב.

```
File Help cadence

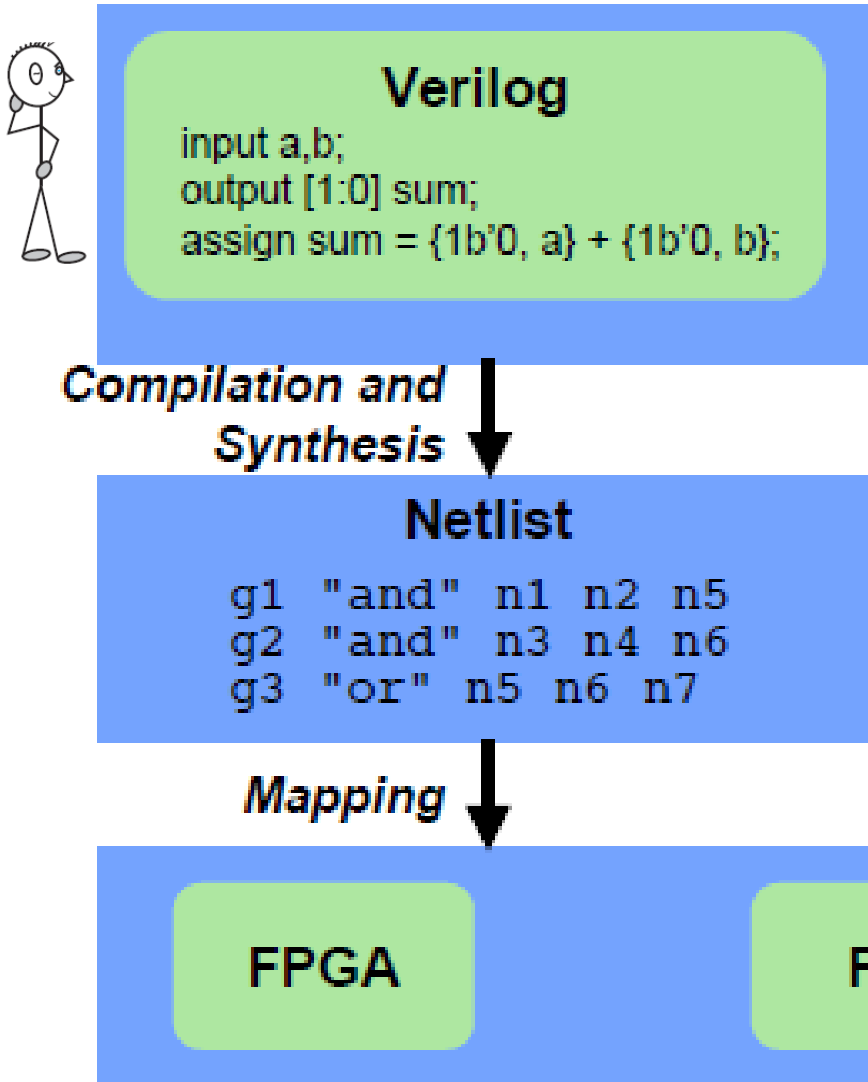
// Generated for: spectre
// Generated on: Jan 30 17:29:16 2009
// Design library name: Lab1B
// Design cell name: 32_32_Array
// Design view name: schematic
simulator lang=spectre
global 0 vdd!

// Library name: Lab1B
// Cell name: lab1B_cell
// View name: schematic
subckt lab1B_cell bit wl
  M4 (bit wl net088 0) NMOS_VTL w=100n l=50n
  M5 (net5 wl 0 0) NMOS_VTL w=100n l=50n
  M3 (net088 lbit_bar 0 0) NMOS_VTL w=100n l=50n
  M2 (lbit_bar lbit 0 0) NMOS_VTL w=100n l=50n
  M1 (0 lbit_bar lbit 0) NMOS_VTL w=100n l=50n
  M0 (lbit 0 0 0) NMOS_VTL w=100n l=50n
  M8 (vdd! lbit_bar lbit vdd!) PMOS_VTL w=200n l=50n
  M9 (net054 lbit_bar vdd! vdd!) PMOS_VTL w=200n l=50n
  M10 (bit net5 net054 vdd!) PMOS_VTL w=200n l=50n
  M7 (lbit_bar lbit vdd! vdd!) PMOS_VTL w=200n l=50n
  M6 (net5 wl vdd! vdd!) PMOS_VTL w=200n l=50n
ends lab1B_cell
// End of subcircuit definition.

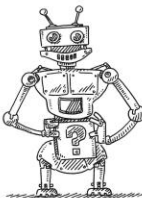
// Library name: Lab1B
// Cell name: RC_BOX
// View name: schematic
subckt RC_BOX IN OUT
  R0 (IN OUT) resistor r=720.00m
  C1 (OUT 0) capacitor c=50.00a
  C0 (IN 0) capacitor c=50.00a
ends RC_BOX
// End of subcircuit definition.

// Library name: Lab1B
// Cell name: 32_32_Array
// View name: schematic
M169 (OUT<0> Add<0> 0 0) NMOS_VTL w=100n l=50n
M167 (OUT<2> Add<0> 0 0) NMOS_VTL w=100n l=50n
M135 (OUT<2> net671 0 0) NMOS_VTL w=100n l=50n
M137 (OUT<1> Add<1> 0 0) NMOS_VTL w=100n l=50n
```

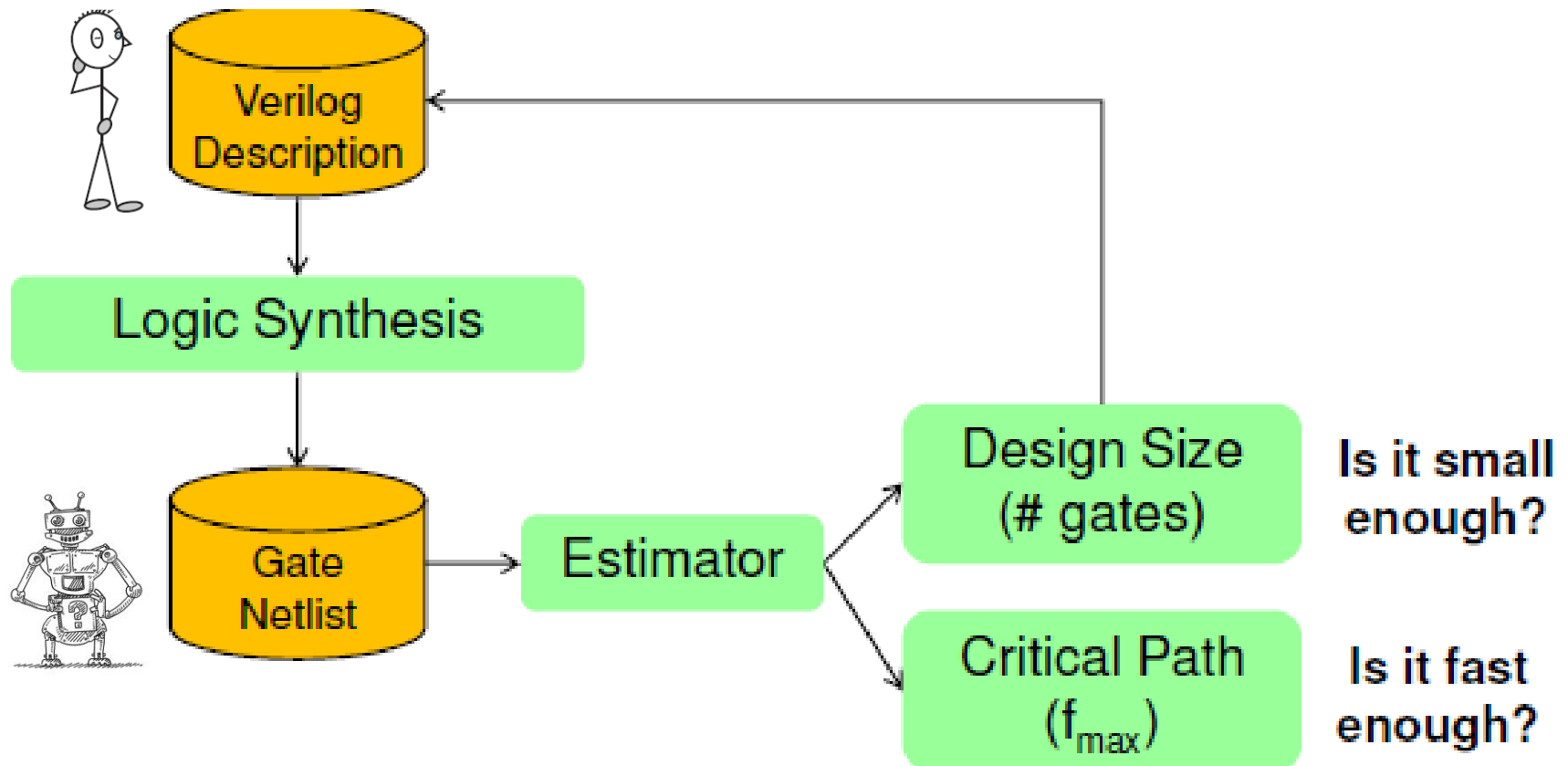
Synthesis and HDLs



- HDL description is compiled into a **netlist**
- **Synthesis** optimizes the logic
- Mapping targets a specific hardware platform



איך אנחנו יודעים שהסינתזה היא בסדר?



Timing



Static Timing Analysis ○

➤ מודל עיכוב דגם

➤ עיכוב מסלול

➤ FALSE מסלול

➤ אילוצי תזמון

Dynamic Timing Analysis ○

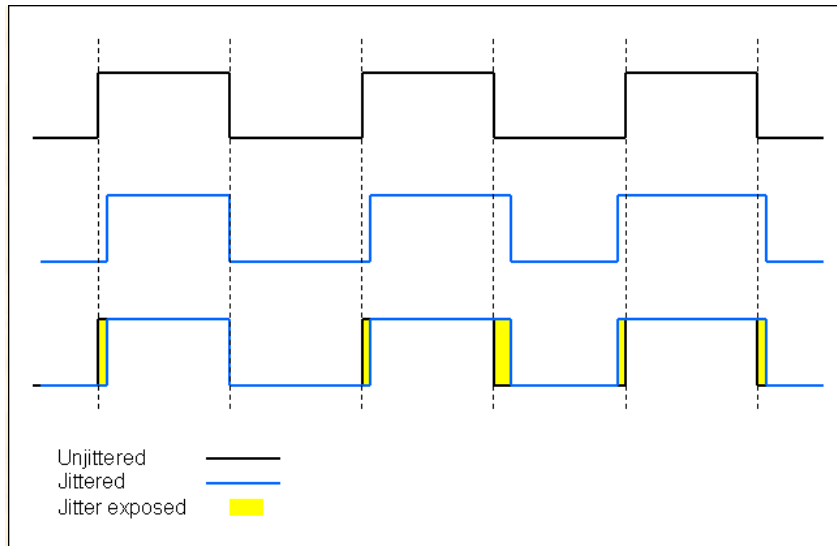
➤ FPGA Design Flow & Timed Simulation

➤ Delay back-annotation with SDF files

Chip Design Specification for Timing Analysis

- Clock Speed = 500 MHz;
- Clock Jitter = +/-0.03ns;
- Voltage Variations = 1.8V+/-0.18 v
- Temperature Variation = 0 -70 0C
- Technology = 180nm

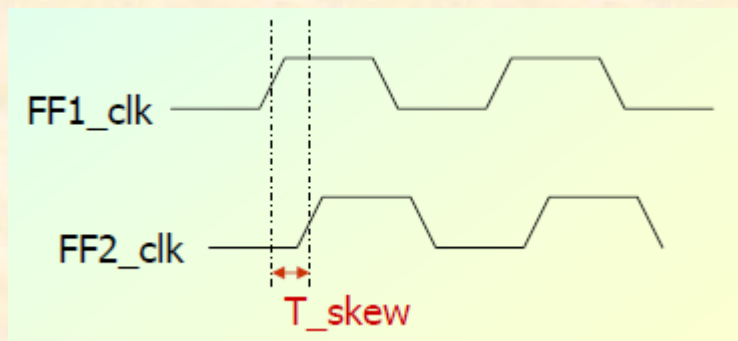
אי-שלמות של שעונים



- **Clock Jitter**
וריאציות של קצוות
רצופות של אות השעון

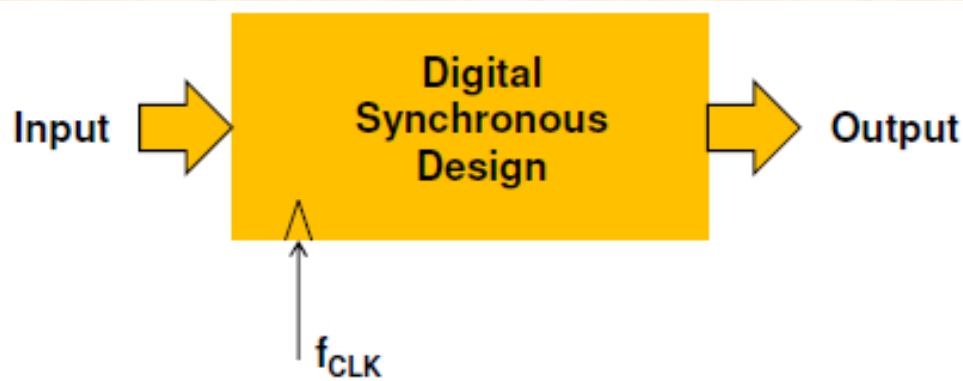
- **Clock Skew**

ההבדל בין הזמנים התפשטות של מעגל השעון נגרמת על ידי buffers או עיכובים אחרים



Static and Dynamic Timing Analysis

- נניח כי אנו רוצים למצוא באופן אוטומטי את תדר השעון המקסימלי של רכיב מסוים



➤ כתוב Verilog

➤ מסונתז בטכנולוגיה
הנתונה

- גישה 1: ניתוח תזמון סטטי = ניתוח טכניקות שימוש

ב- netlist להגדרת $f_{clk,max}$

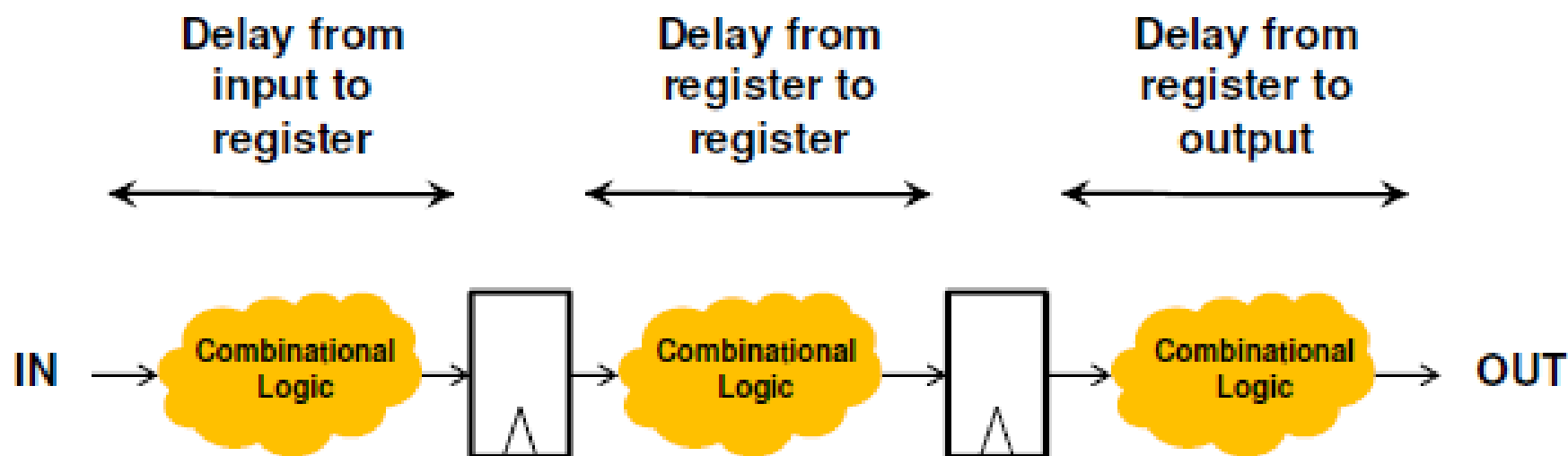
- גישה 2: ניתוח תזמון דינמי = סימולציה ושימוש בקלטים נבחרים כדי למדוד את המסלול האיטי ביותר

Static and Dynamic Timing Analysis

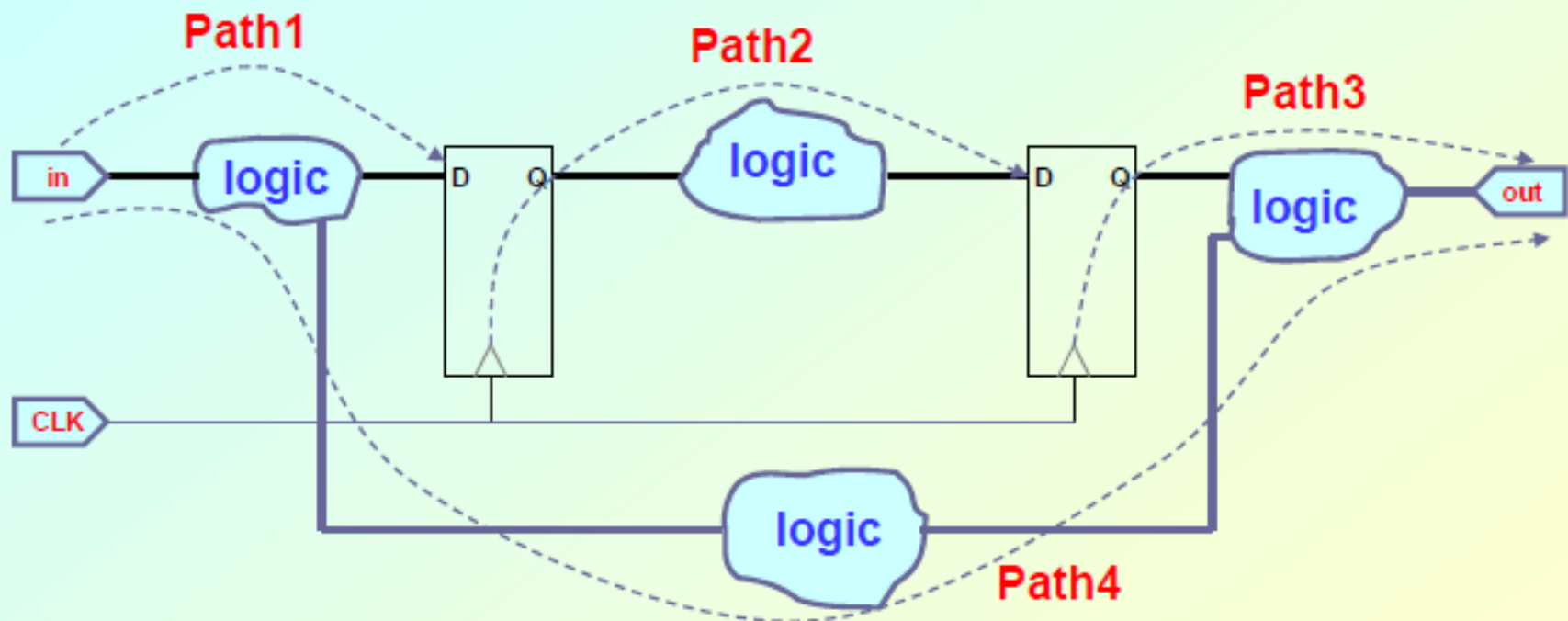
- **ניתוח תזמון סטטי** נראה אידיאלי: הוא נעשה על ידי כלי שמתחיל מ-netlist ומופעל באופן אוטומטי. עם זאת, ניתוח תזמון סטטי עשוי לדווח על תוצאות פסימיות ולדווח עיכובים שלא יתרחשו במהלך הריצות.
- **ניתוח תזמון דינמי** נחמד כי אנחנו יכולים לשלב את זה עם סימולציות אשר אנו עושים עבור האימות הפונקציונלי עם זאת, ניתוח תזמון דינמי יכול להיות אופטימי מדי, במיוחד אם אנחנו לא הצלחנו לדמות המקרה הגרוע ביותר (אשר עשוי להיות קשה לחזות ב-netlist מורכב)

Static Timing Analysis

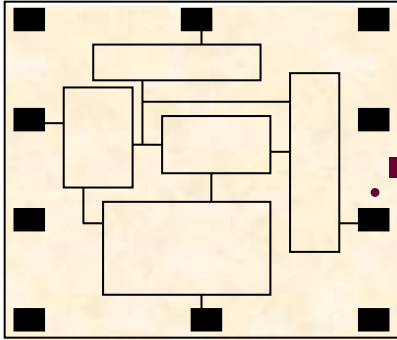
- STA מחפש את הגרוע ביותר בין העיכובים הבאים
בשבב



Timing and Paths



תכנון פיזי -- Physical Design



1. המעגלים של כל רכיב מומרים לייצוג גיאומטרי.

2. תוצאת ההמרה של netlist לתוך ייצוג גיאומטרי נקראת layout.

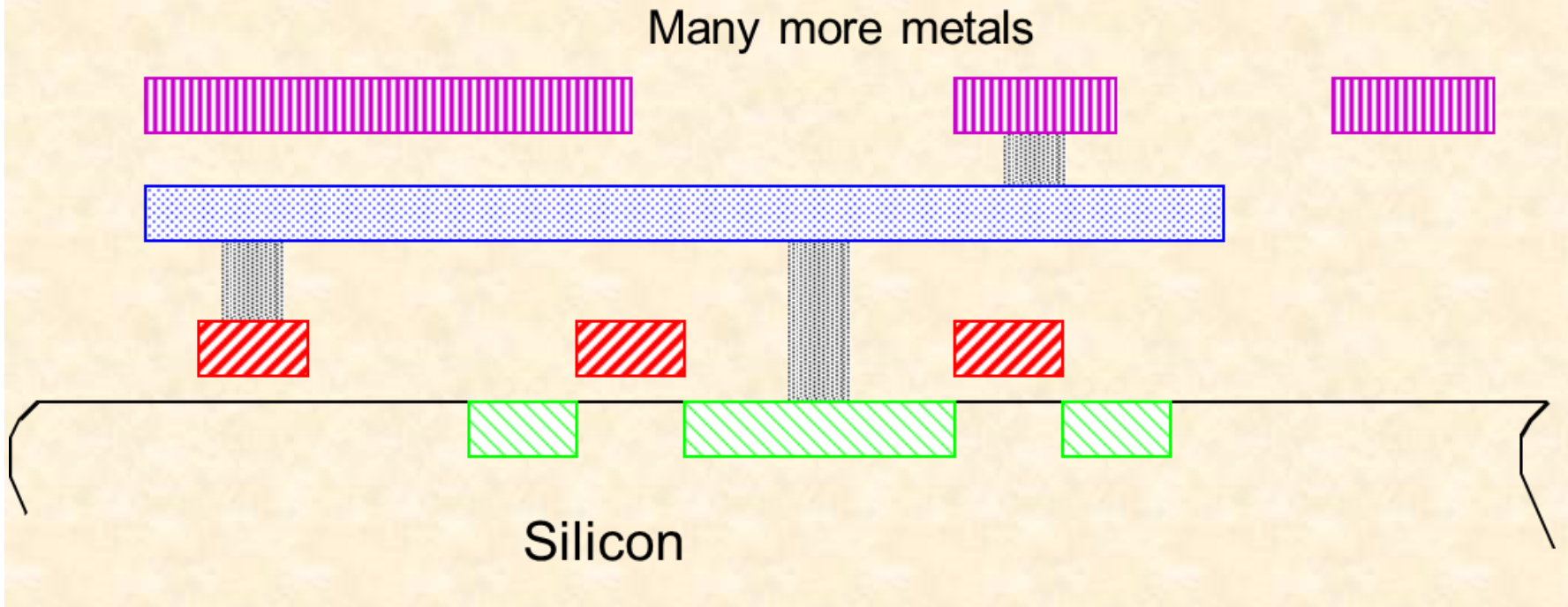
3. קשרים בין המרכיבים השונים באים לידי ביטוי גם בצורות גיאומטריות.

4. מידות מדוייקיות תלויות בכללי ייצור

5. בדיקות אימות שונות מבוצעות על ה layout במהלך תכנון פיזי.

ייצור CMOS

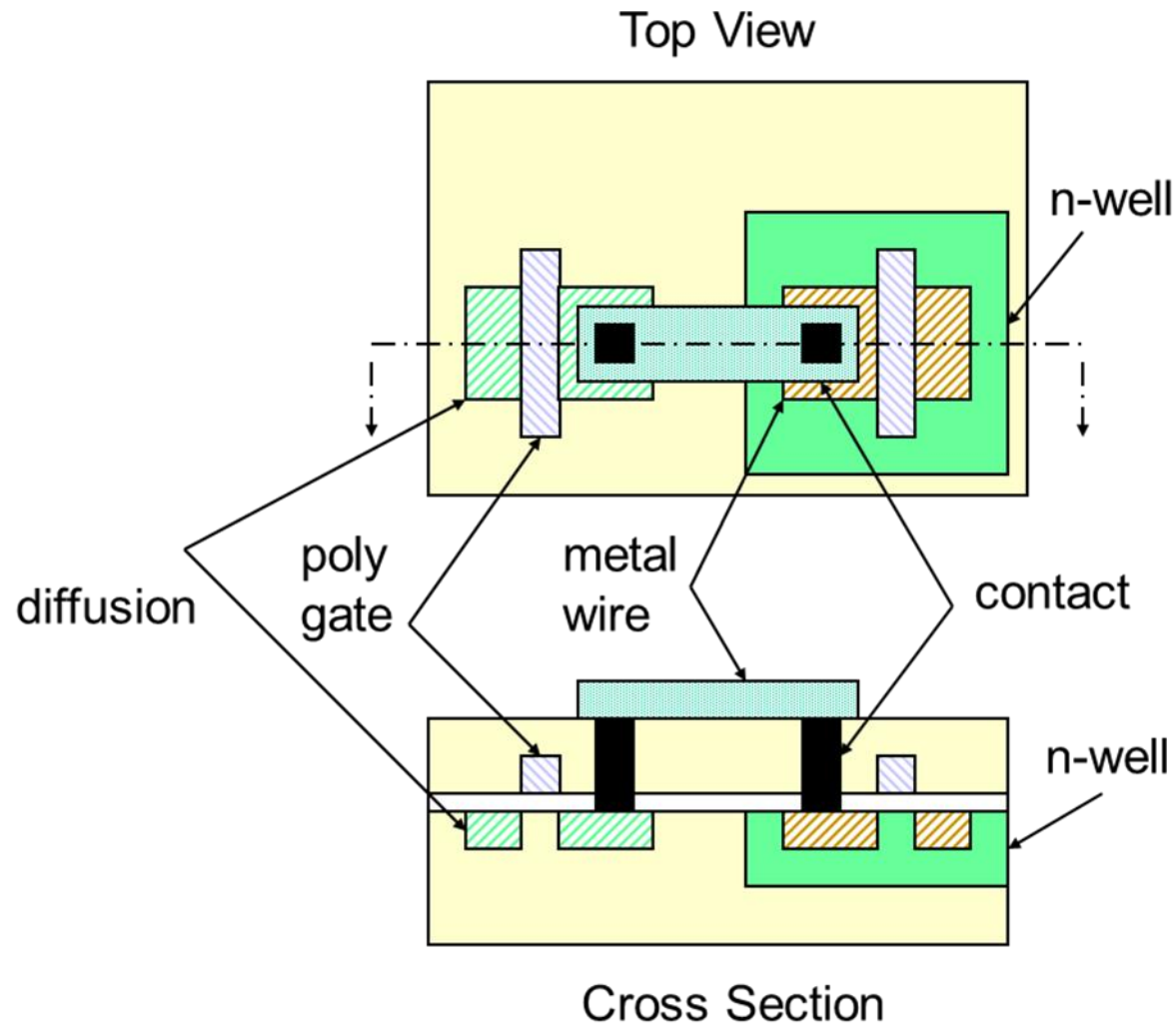
- טרנזיסטורים CMOS מיוצרים על פרוסות סיליקון
- ליתוגרפיה דומה לתהליך הדפוס
- בכל שלב, חומרים שונים מופקדים או חרוטים



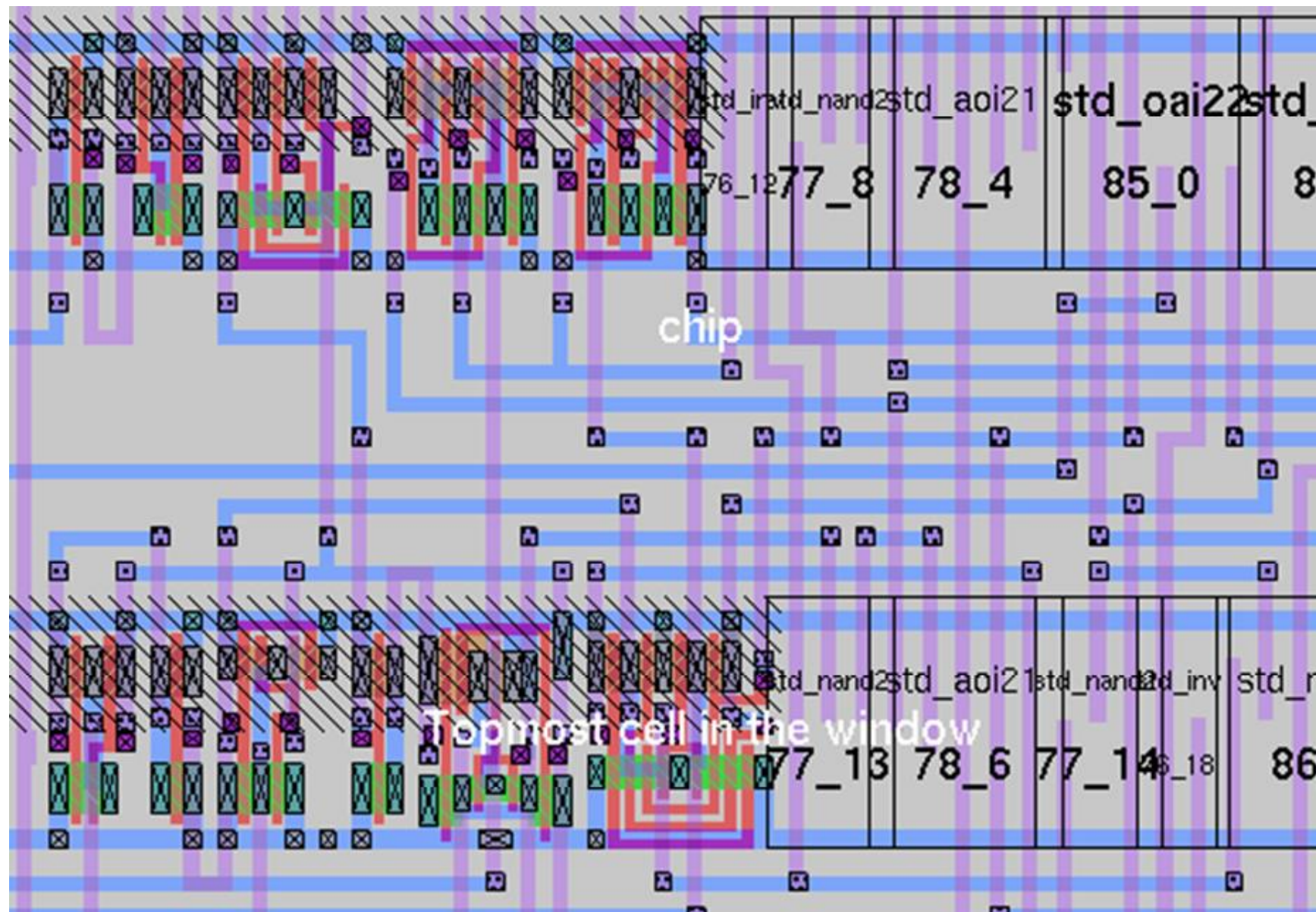
Layout

- שבבים מוגדרים בעזרת סט של מסכות
- הגיאומטריה נדרשת להגדרת ולהתחברות של אלמנטים במעגל
- מידות מינימליות של מסכות קובעות בגודל טרנזיסטור (ולכן מהירות, עלות והספק)
- גודל הטרנזיסטור שווה למרחק בין המקור והיציאה שווה לרוחב מינימלי של polysilicon

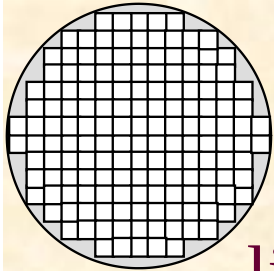
Layout



Controller Layout



ייצור



1. התהליך כולל:

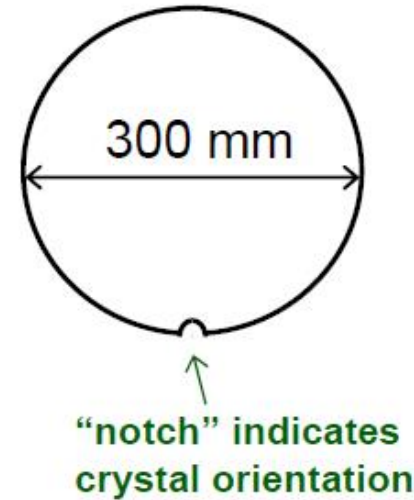
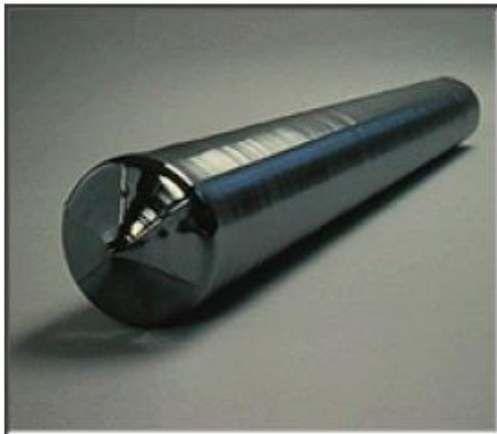
lithography, polishing, deposition, diffusion, etc

2. תהליך הייצור כולל מספר שלבים ומחייב מסכות שונות.

3. לפני ששבב מיוצר בכמויות, אב טיפוס מורכב ונבדק.

Integrated Circuit Fabrication

- **המתרה:** ייצור המוני (כלומר ייצור בו זמני) של שבבים רבים כל אחד מעגל (למשל מעבד או זיכרון) המכיל מיליונים או מיליארדים של טרנזיסטורים
- **שיטה:** ייצור סרטים דקים של מוליכים למחצה, מתכות ומבודדים בדומה לדפסה (ליתוגרפיה).



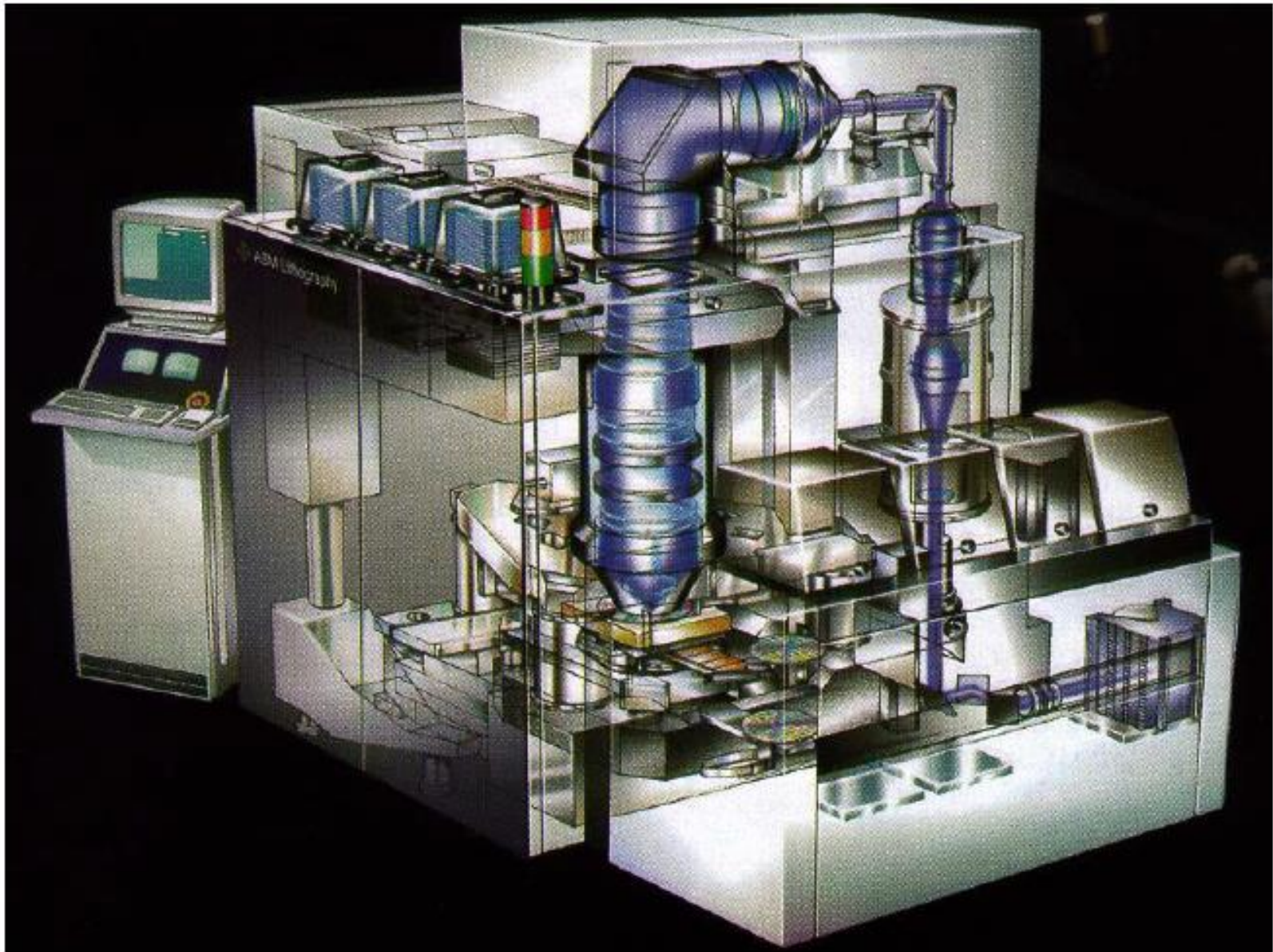
Typical wafer cost: \$50

Sizes: 150 mm, 200 mm, 300 mm diameter

Wafer Production



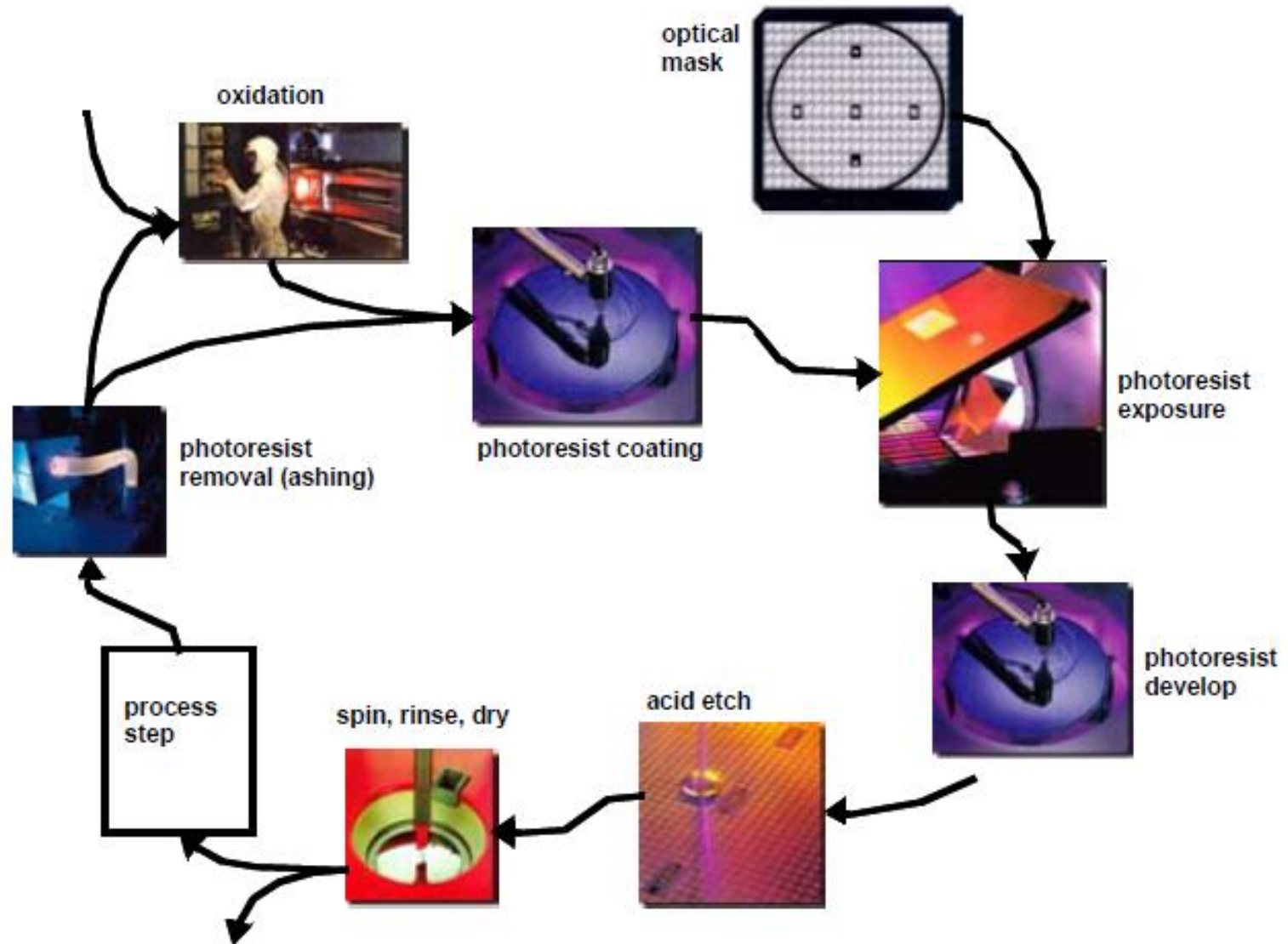
Commercial stepper



Wafer production

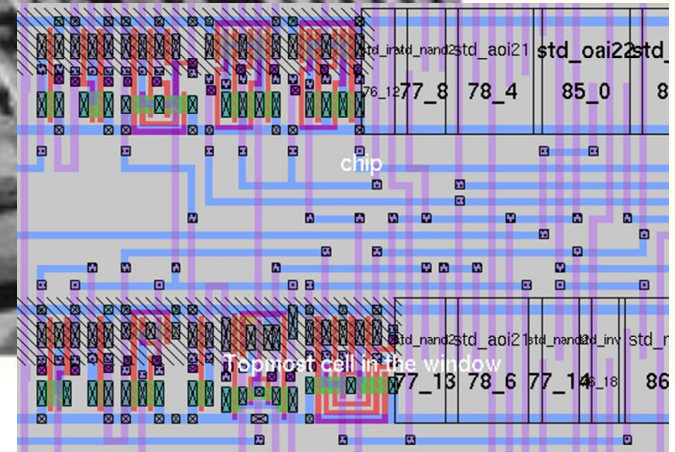
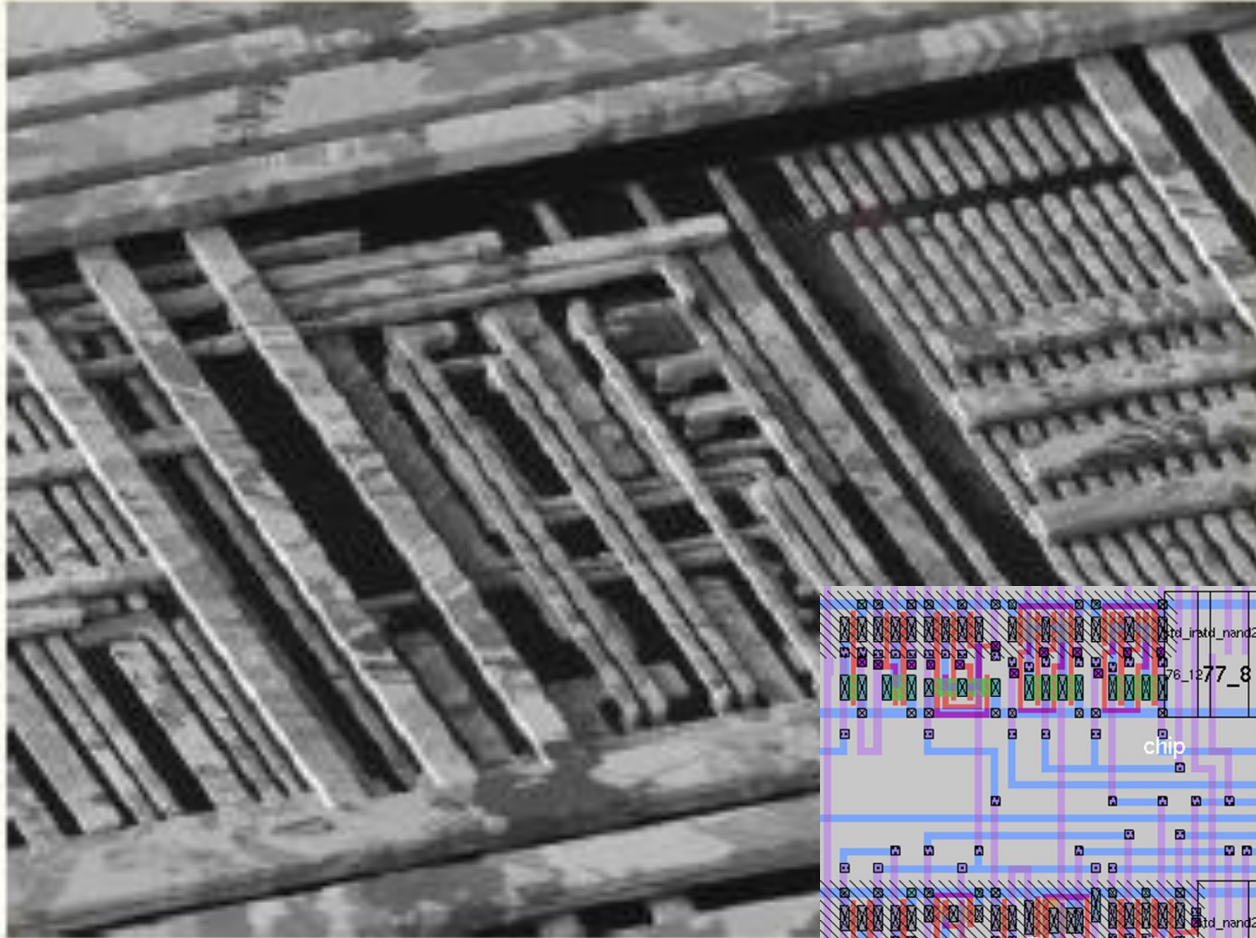


The photo-lithographic process



תהליך נחושת

Copper Damascene Interconnect (Intel)

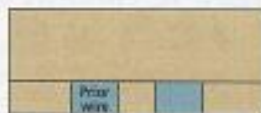


Advanced metalization

IBM Technology (in Rabaey, Digital Integrated Circuits, 2nd ed.)
First commercial Copper process
(0.12μ)

Dual damascene IC process

• Oxide deposition



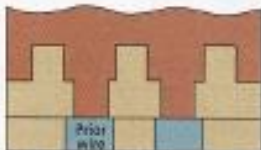
• Stud lithography and reactive ion etch



• Wire lithography and reactive ion etch



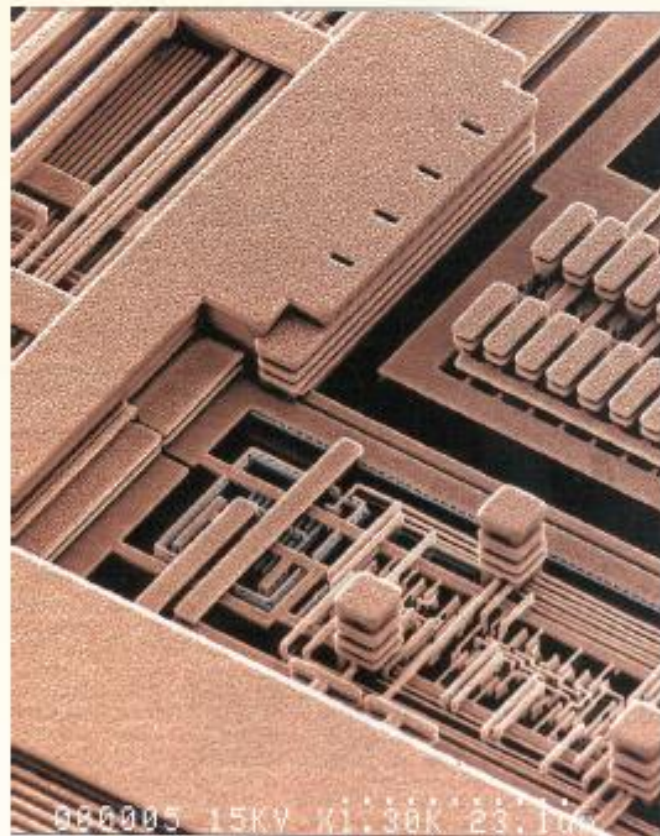
• Stud and wire metal deposition



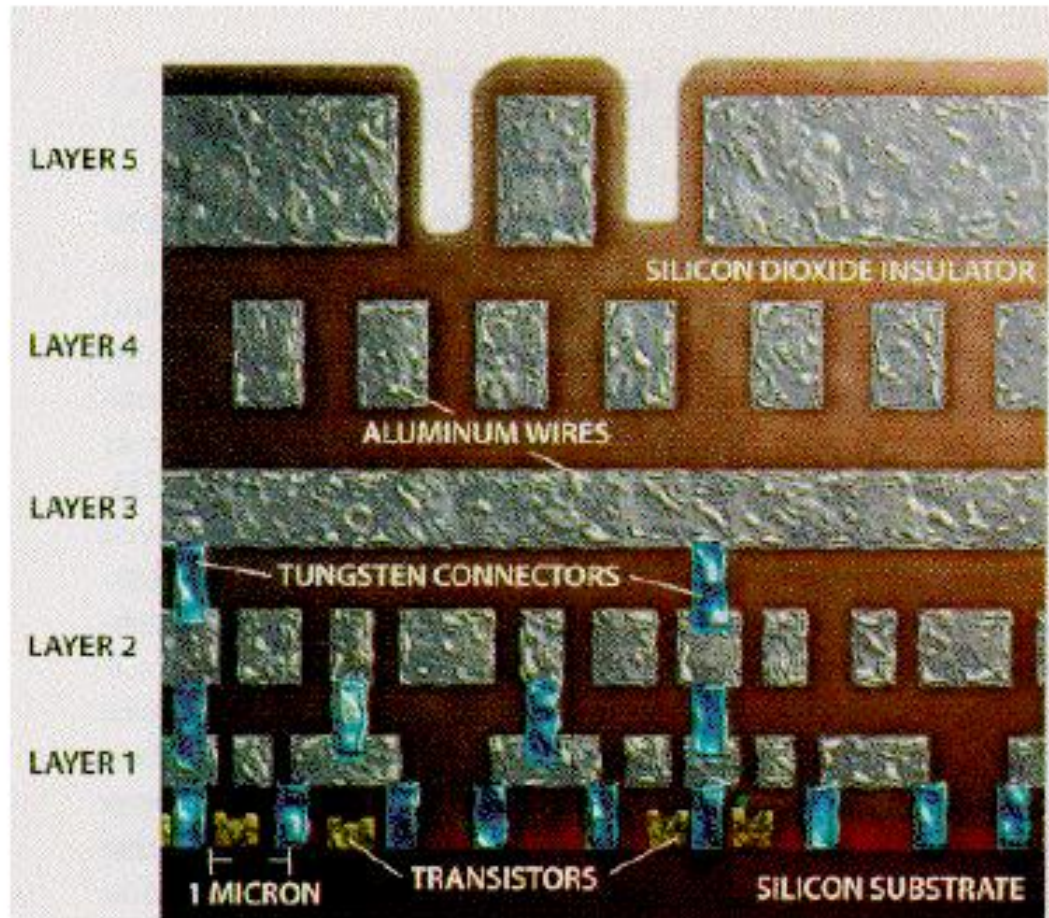
• Metal chemical-mechanical polish



Source: IBM Corp.

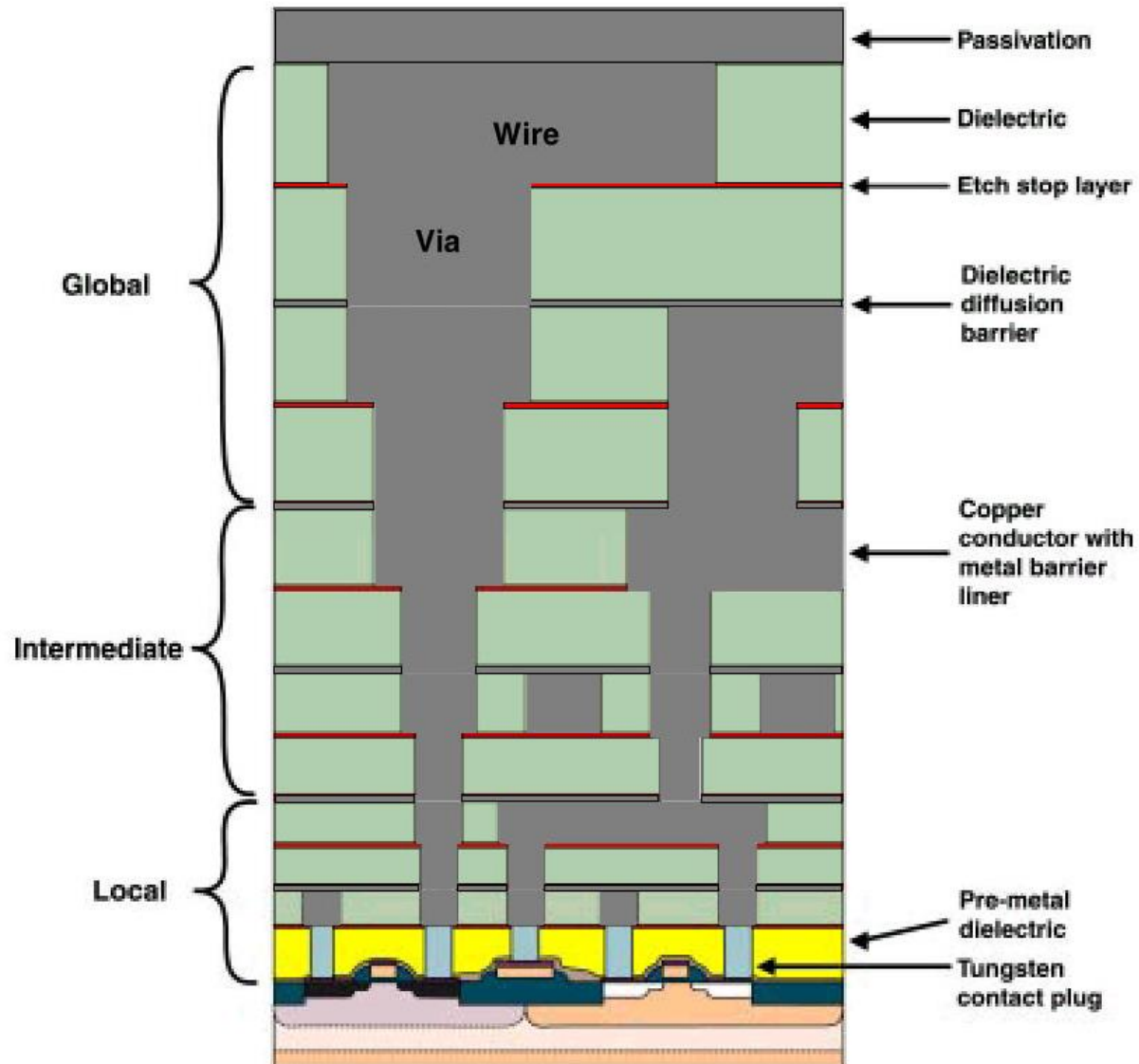


שכבות בשבב



5-layer cross-section of chip

Cross section

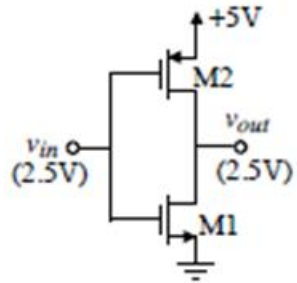


אריזה ובדיקות סופיות

1. אריזה - הרכבה של השביבים על מעגלים מודפסים
(PCB (Printed Circuit Board או Multi-Chip
(Module מלמ"ק)
2. כל שבב ארוז ואז נבדק כדי להבטיח שהוא עונה על
כל מפרטי העיצוב ומתפקד כראוי.

העברה לייצור

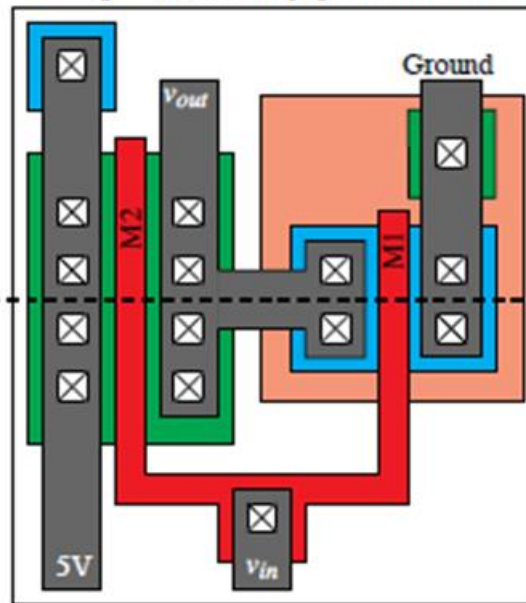
CIRCUIT



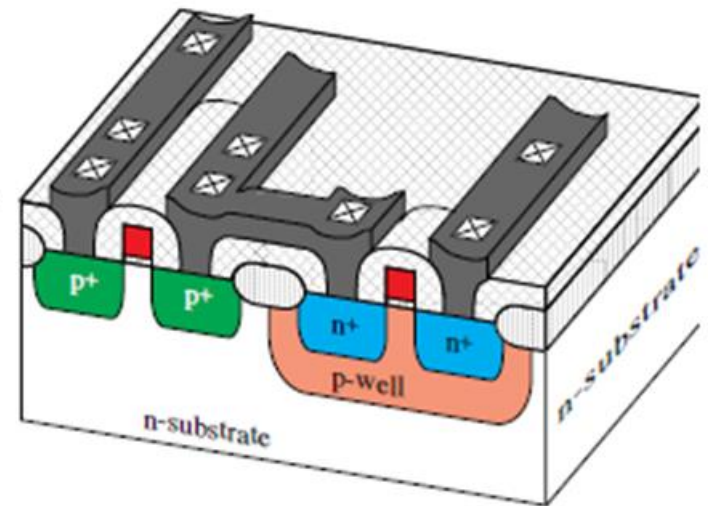
031113-01

LAYOUT

Blue	Green	Black	Red	Orange	White
n ⁺	p ⁺	Metal	Poly	p-well	n-substrate



FABRICATION



שלבים שונים של תכנון

